Orchfile Grammar Reference

Orchfile Specification

This document contains the formal grammar for Orchfile in W3C EBNF notation, followed by railroad (syntax) diagrams for the key production rules. The canonical source is `grammar.ebnf` in the repository root.

Contents

1 EBNF Grammar **2**

1.1 Document Structure . . . 2
1.2 Top-level Declarations . . . 2
1.3 Directives . . . 2
1.3.1 Overlay Composition . . . 2
1.3.2 Execution Mode (mutually exclusive per service) . . . 2
1.3.3 Container-only Directives . . . 2
1.3.4 Host-only Directives . . . 3
1.3.5 Common Directives . . . 3
1.4 Terminals . . . 4

2 Railroad Diagrams **6**

3 Constraints **8**

1 EBNF Grammar

1.1 Document Structure

```
/* Directives start at column 0. Trailing whitespace on a line is not part of any
    value. */
orchfile    ::= bom? line ( newline line )*   /* the final line ending is optional */
line        ::= blank | comment | ( version | arg | service_decl | directive ) ws?
blank       ::= ws*
comment     ::= ws* '#' any*
bom         ::= #xFEFF   /* optional UTF-8 byte-order mark; ignored */
```

1.2 Top-level Declarations

```
version     ::= 'ORCH_VERSION' ws version_number
arg         ::= 'ARG' ws identifier '=' value
service_decl ::= 'SERVICE' ws service_name
```

1.3 Directives

```
directive   ::= mode_directive
              | container_directive
              | host_directive
              | common_directive
              | clear_directive
```

1.3.1 Overlay Composition

```
/* Overlay composition - resets a list-type directive before merge */
clear_directive
            ::= 'CLEAR' ws clearable_target

clearable_target
            ::= 'ENV' | 'ENV_FILE' | 'PUBLISH' | 'VOLUME'
              | 'REQUIRES' | 'AFTER'
```

1.3.2 Execution Mode (mutually exclusive per service)

```
/* Execution mode - mutually exclusive per service */
mode_directive
            ::= 'FROM' ws image_ref
              | 'RUN' ws command_string
```

1.3.3 Container-only Directives

```
/* Container-only */
container_directive
            ::= 'ENTRYPOINT' ws command_string
              | 'CMD' ws command_string
              | 'PUBLISH' ws ( host_address ':' )? port ':' port
              | 'VOLUME' ws volume_source ':' volume_destination ( ':' volume_mode )?
```

1.3.4 Host-only Directives

```
/* Host-only */
host_directive
            ::= 'USER' ws user_name
              | 'STOP' ws command_string
              | 'RELOAD' ws command_string
```

1.3.5 Common Directives

```
/* Common - environment */
env_directive
            ::= 'WORKDIR' ws path
              | 'ENV' ws env_key '=' value
              | 'ENV_FILE' ws path

/* Common - dependencies */
dependency_directive
            ::= 'REQUIRES' ws service_name ( ws service_name )*
              | 'AFTER' ws service_name ( ws service_name )*

/* Common - health */
health_directive
            ::= 'HEALTHCHECK' ws ( url | command_string )
              | 'READINESS_TIMEOUT' ws duration

/* Common - lifecycle */
lifecycle_directive
            ::= 'ONESHOT' ws bool
              | 'DISABLED' ws bool
              | 'RECREATE' ws recreate_policy

/* Common - restart */
restart_directive
            ::= 'RESTART' ws restart_policy
              | 'RESTART_DELAY' ws duration
              | 'START_LIMIT_BURST' ws integer
              | 'START_LIMIT_INTERVAL' ws duration

/* Common - timeouts */
timeout_directive
            ::= 'TIMEOUT_START' ws duration
              | 'TIMEOUT_STOP' ws duration

/* Common - resources */
resource_directive
            ::= 'MEMORY' ws memory_size
              | 'CPUS' ws number
              | 'CPU_QUOTA' ws percentage
              | 'LIMIT_NOFILE' ws integer
              | 'LIMIT_NPROC' ws integer
              | 'TASKS_MAX' ws integer
              | 'IO_WEIGHT' ws integer   /* 10-1000 */

/* Common - logging */
logging_directive
            ::= 'STDOUT' ws path
```

```
              | 'STDERR' ws path

common_directive
            ::= env_directive
              | dependency_directive
              | health_directive
              | lifecycle_directive
              | restart_directive
              | timeout_directive
              | resource_directive
              | logging_directive
```

1.4 Terminals

```
/* Terminals */
service_name ::= letter ( letter | digit | '-' )*   /* max 63 chars */
identifier   ::= ( alpha | '_' ) ( alpha | digit | '_' )*   /* ARG names */
env_key      ::= identifier
user_name    ::= ( alpha | digit | '_' ) ( alpha | digit | '_' | '-' | '.' )*

volume_source      ::= volume_part   /* host path or named volume */
volume_destination ::= volume_part   /* container path */
volume_part ::= ( visible - ':' ) ( ( any - ':' )* ( visible - ':' ) )?
volume_mode ::= 'ro' | 'rw'          /* rw if omitted */

/* Free text. It never starts or ends with whitespace. */
text         ::= visible ( any* visible )?
image_ref    ::= text
command_string ::= text
path         ::= text
value        ::= ( any* visible )?   /* after '='; may be empty */
url          ::= ( 'http://' | 'https://' ) text

/* var_ref and var_escape may appear anywhere in the value of a directive inside a
    SERVICE block. Each value must match its rule after expansion (see SPEC.md). */
var_ref      ::= '${' identifier '}'   /* identifier MUST be a declared ARG */
var_escape   ::= '$${'                 /* a literal '${'; never starts a var_ref */

bool         ::= 'true' | 'false'
restart_policy ::= 'no' | 'always' | 'on-failure'
recreate_policy ::= 'always' | 'never'

duration     ::= integer ( 's' | 'm' )
memory_size  ::= integer ( 'K' | 'M' | 'G' )
percentage   ::= integer '%'
number       ::= integer ( '.' digit+ )?
integer      ::= digit+
port         ::= digit+   /* 1-65535 */
version_number ::= integer '.' integer '.' integer ( '-' prerelease )?
prerelease   ::= ( letter | digit | '-' | '.' )+   /* e.g. rc, beta.1 */

host_address ::= ipv4 | ipv6 | hostname
ipv4         ::= octet '.' octet '.' octet '.' octet
octet        ::= digit+                          /* 0-255 */
ipv6         ::= ( hex_digit | ':' | '.' )+      /* RFC 4291 text form, without
    brackets */
hostname     ::= label ( '.' label )*            /* RFC 1123 */
```

```
label       ::= alnum ( ( alnum | '-' )* alnum )?   /* max 63 chars */
alnum       ::= alpha | digit
hex_digit   ::= digit | [a-fA-F]

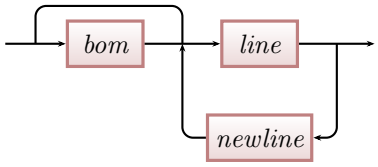
letter      ::= [a-z]
alpha       ::= [a-zA-Z]
digit       ::= [0-9]
ws          ::= [ #x9]+
newline     ::= #xA | #xD #xA
any         ::= #x9 | [#x20-#x7E] | [#xA0-#xD7FF] | [#xE000-#x10FFFF]   /* no control
    characters except tab; no surrogates */
visible     ::= any - [ #x9]
```

Note: The grammar describes each value *after* variable expansion. `var_ref` (`${name}`) may appear anywhere in the value of a directive inside a SERVICE block, and `var_escape` (`$${`) stands for a literal `${`. With `ARG port=8080`, `PUBLISH ${port}:80` is checked as `PUBLISH 8080:80`. Every name must be a declared ARG; there are no built-in variables.

2 Railroad Diagrams

The following diagrams visualize the key production rules. Rounded boxes are *terminals* (literal text), rectangular boxes are *non-terminals* (references to other rules).

orchfile



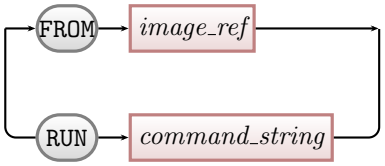
arg



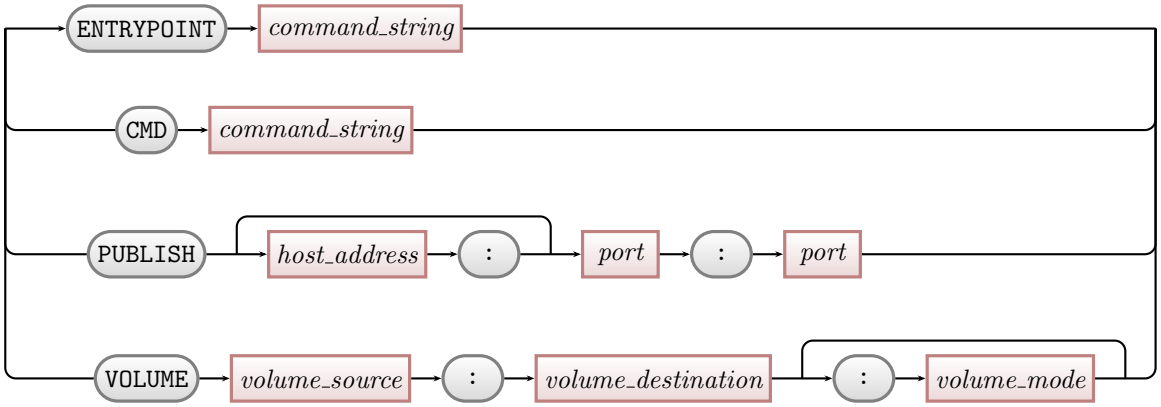
service_decl



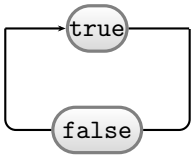
mode_directive



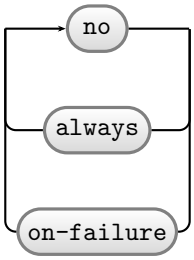
container_directive



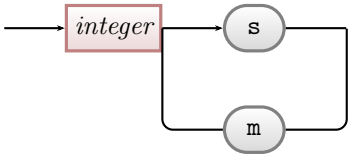
bool



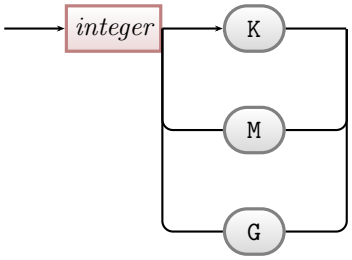
restart_policy



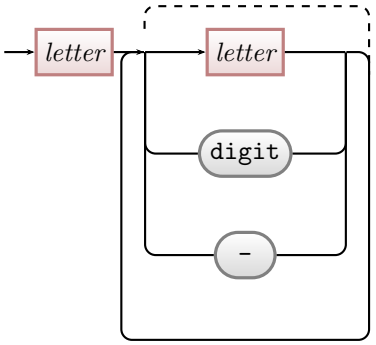
duration



memory_size



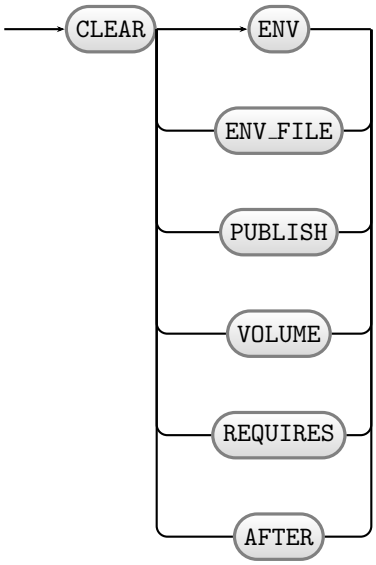
service_name (max 63 characters)



var_ref



clear_directive



3 Constraints

The following constraints are enforced by the parser and cannot be expressed in the EBNF grammar alone:

C1: FROM XOR RUN A service must specify exactly one of `FROM` (container mode) or `RUN` (host mode). Both or neither is an error.

C2: Container-only directives `ENTRYPOINT`, `CMD`, `PUBLISH`, `VOLUME` are only valid with `FROM`.

C3: Host-only directives `USER`, `STOP`, `RELOAD` are only valid with `RUN`.

C4: Dependency acyclicity `REQUIRES` and `AFTER` edges must form a directed acyclic graph (DAG). Cycles are a parse error.

C5: Declared variables Every `${name}` must name an `ARG` declared in one of the merged files.

C6: One entry per key Within a service, each container port is published at most once (`PUBLISH`) and each destination is mounted at most once (`VOLUME`).

C7: No repeats within a file Within one file, a service sets each scalar directive and each `ENV` key at most once. Across files, overlays replace values.

Numeric ranges noted in the grammar's comments (ports 1–65535, IPv4 octets 0–255, hostname labels and service names up to 63 characters, `IO_WEIGHT` 10–1000) are also enforced by the parser.